

INVENTORY TRACKER

Project Specification Document

Software Requirements Specification | SRS-IT-2026-001

Version 1.0 | June 2026 | Status: Approved

A complete engineering reference covering architecture, data models, APIs, security, testing, and operational requirements for a multi-tenant warehouse inventory management SaaS.

Field	Value
Project Name	Inventory Tracker
Document ID	SRS-IT-2026-001
Version	1.0.0
Status	Approved
Author	Riya Gupta (riya.gupta@futuresmart.ai)
Last Updated	June 28, 2026
Confidentiality	Internal — Engineering Only
Stack	Next.js 16 React 19 MongoDB TypeScript Vercel

Executive Summary

What Inventory Tracker is, why it exists, and what it delivers

Inventory Tracker is a multi-tenant, cloud-hosted inventory management SaaS built for small-to-medium businesses operating across multiple physical warehouses. It replaces spreadsheets and disconnected tools with a unified platform for tracking stock inflows, outflows, transfers, aging, and alerts in real time.

Metric	Detail
41	API Endpoints across 13 resource groups
11	MongoDB + Supabase data models
2	User roles: Super Admin & Warehouse Manager
5	Alert types: Dead Inventory, Aging, Low Stock, Expiry, Capacity
2	Daily cron jobs: aging update + alert generation
Next.js 16	React 19, TypeScript, Vercel serverless

Problem Statement

Warehouse operators at growing companies face a critical gap: stock aging silently into dead inventory, low-stock situations going unnoticed until orders fail, and no visibility across locations without manual reconciliation. Spreadsheet-based tracking breaks down when a business operates 3+ warehouses with different managers, product categories, and aging timelines — leading to financial write-offs, stockouts, and audit risk from no movement history.

Proposed Solution

- Tracks every stock batch from warehouse entry through exit, including transfers between locations
- Automatically calculates aging (days since entry) and progresses stock through Healthy → At Risk → Dead
- Generates proactive alerts via automated cron jobs and mutation-triggered checks
- Enforces role-based access so warehouse managers only see their assigned locations
- Provides a photo-verified audit trail for all stock movements

Business Goals

- Reduce dead inventory write-offs by 30%+ by surfacing aging issues before they become losses
- Enable a single admin to manage unlimited warehouses from one dashboard
- Zero-cost onboarding — no credit card required — to drive SMB adoption
- Complete movement audit trail for supplier disputes and compliance requirements

Background & Vision Statement

Why this project was built and the north-star that guides every decision

Existing Pain Points

ID	Severity	Pain Point
P-1	Critical	No aging visibility — stock sits unnoticed until physically counted
P-2	Critical	No cross-warehouse view — each location managed in isolation
P-3	High	No alerts — low stock discovered only when an order fails
P-4	High	No photo audit trail — disputed deliveries with no evidence
P-5	High	No role-based access — all staff see everything
P-6	High	No transfer records — stock moves leave no audit trail

Why Build vs. Buy

Existing SaaS solutions (Zoho Inventory, inFlow, Fishbowl) were evaluated and ruled out: cost-prohibitive for SMBs (Zoho starts at Rs.4,000/month), over-engineered for manufacturing ERP workflows, dead stock tracking is a premium add-on (not core), and their user models don't match the warehouse-manager-per-location access pattern required.

Vision Statement

"To be the inventory brain that every warehouse operator wishes they had — one that watches every batch, learns every aging pattern, and speaks up before silence costs money."

Objectives Summary

Category	Objective
Business	Reduce dead inventory write-offs by 30%+ within 6 months of adoption
Business	Enable zero-cost onboarding; no credit card, no trial expiry
Technical	All data isolated by companyId — no cross-tenant leakage at any layer
Technical	JWT stored in HttpOnly cookies — zero client-side token exposure
Technical	Stock entry/exit/transfer use MongoDB transactions — no partial writes
Performance	Dashboard load <2s cold start; all API endpoints <500ms at p95
Security	Passwords bcrypt-hashed (salt rounds >= 10); plaintext never stored
Security	Cron endpoints secured by bearer token; not browser-accessible

Category	Objective
UX	Complete onboarding in <=5 steps, <10 minutes, no dead ends
UX	Full dark mode; responsive from 375px mobile to 4K desktop

Scope & Stakeholders

In Scope — v1.0

- Multi-tenant company onboarding wizard (5 steps)
- JWT authentication with email/password login and password reset
- User invitation system for warehouse managers
- Product catalogue management (CRUD, soft archive)
- Multi-warehouse management (CRUD, capacity tracking)
- Stock entry with photo upload (Cloudinary)
- Stock exit with customer/order reference
- Stock transfer between warehouses with transport details
- Batch-level stock tracking with unique batch IDs and parentBatchId transfer chain
- Automated aging: Healthy (0-119 days) -> At Risk (120-179) -> Dead (180+ days)
- 5 alert types: dead inventory, aging, low stock, expiry warning, warehouse full
- Alert lifecycle: open -> acknowledged -> resolved / dismissed
- In-app notifications per user + critical email alerts via Gmail SMTP
- Aging stock report with healthy/at-risk/dead breakdown
- Inventory dashboard with health score, warehouse snapshot, activity feed
- Role-based access: Super Admin and Warehouse Manager
- Audit log for all entity mutations
- Dark mode support + responsive design
- Vercel Analytics integration + health check endpoint

Out of Scope — v1.0 (Planned for v2+)

- Payment processing or subscription billing
- Native mobile app (iOS / Android)
- SMS notifications | OAuth / SSO | MFA
- Barcode / QR code scanning
- ERP / accounting integration (QuickBooks, Tally)
- Supplier management, purchase orders, sales orders
- AI-powered demand forecasting or automated reordering
- Custom report builder (XYZ analysis, valuation, damage reports — UI placeholders only)
- Real-time collaborative editing (WebSockets)
- On-premises / self-hosted deployment

Stakeholders

Role	Name/Team	Responsibility	Status
Product Manager	Riya Gupta	Requirements, roadmap, prioritization	Active
Engineering Manager	TBD	Technical direction, release sign-off	Active

Role	Name/Team	Responsibility	Status
Frontend Engineers	1-2 engineers	Next.js pages, components, forms, theming	Active
Backend Engineers	1-2 engineers	API routes, data models, cron, auth	Active
QA Team	TBD	Functional testing, regression, edge cases	Review
DevOps	Vercel (managed)	Deployment, cron scheduling, env config	Platform
End Users — Super Admin	Ops managers / business owners	Company-wide inventory visibility	Consulted
End Users — Warehouse Mgr	Site warehouse staff	Daily stock recording and transfers	Consulted
MongoDB Atlas	Cloud DB vendor	Managed MongoDB hosting & backups	Vendor
Supabase	Permission store	PostgreSQL for warehouse permission table	Vendor
Cloudinary	Image CDN	Photo storage for stock entry/exit	Vendor
Vercel	Hosting platform	Serverless functions, cron, analytics	Vendor

User Personas

Two primary personas drive all design and prioritization decisions

Persona 1 — Super Admin: Ravi Mehta

Field	Detail
Age	42
Location	Mumbai, India
Role	Operations Manager, mid-size building materials distributor (4 warehouses)
Tech Level	Moderate — comfortable with web apps and spreadsheets, not a developer
Primary Device	Desktop (primary), laptop (travel)
Goals	Single dashboard for all 4 sites; advance warning on aging; assign managers without IT help
Pain Points	Surprised by end-of-quarter dead stock write-offs; no excess-to-redistribute visibility; inconsistent WhatsApp updates from managers
Typical Journey	1) Check dashboard health score and alerts 2) Review and acknowledge critical alerts 3) Check warehouse utilization for near-full sites 4) Open aging report — review dead/at-risk by value 5) Initiate transfer to balance load 6) Invite new warehouse manager via email

Persona 2 — Warehouse Manager: Priya Singh

Field	Detail
Age	28
Location	Pune, India
Role	Warehouse Supervisor, manages Pune site (1 of 4 warehouses)
Tech Level	Basic — uses phone daily; comfortable with simple forms and photo uploads
Primary Device	Mobile (primary on floor), tablet (office desk)
Goals	Log incoming stock quickly with photo proof; record dispatch with order reference; know when stock runs low before it becomes a problem
Pain Points	Paper entry books get lost; WhatsApp photos have no systematic record; blamed for discrepancies with no evidence to dispute; can't see which items are aging
Typical Journey	1) Truck arrives — Record Entry on mobile, select product + warehouse 2) Take entry photo, enter quantity and date 3) Submit — batch logged with timestamp and photo 4) Check notifications for low-stock alerts 5) Customer pickup — record exit with order reference 6) Review her warehouse aging report, flag batches to admin

Functional Requirements

23 requirements — each with description, dependencies, and testable acceptance criteria

Priority levels: **Critical** — system unusable without it | **High** — core feature, significant value loss if missing | **Medium** — important with workaround | **Low** — nice to have

FR-01	User Authentication (Email/Password Login)	Critical
Description	Users authenticate with email and password. Successful login stores a signed JWT as an HttpOnly cookie (auth-token, 7-day expiry). Failed attempts return a generic error without revealing whether the email exists. Password show/hide toggle on input.	
Dependencies	MongoDB, JWT_SECRET env var, bcryptjs	
Acceptance Criteria	- Valid credentials -> 200 + auth cookie + redirect to /dashboard - Wrong password -> 401 generic error, no cookie set - Unknown email -> 401 same message (no email enumeration) - needsOnboarding flag -> redirect to /onboarding - Cookie is HttpOnly, Secure in production, SameSite=Lax - Token expires after 7 days; expired token -> redirect to login	

FR-02	Company Onboarding Wizard (5-Step)	Critical
Description	New companies complete a 5-step guided setup: (1) Admin credentials, (2) Company config (name, industry, currency, concerns), (3) Warehouse setup (repeatable, min 1), (4) Setup preference (Quick/Custom), (5) Product categories (repeatable, min 1). Creates SystemConfig, User, Warehouse(s), and ProductCategory records atomically via MongoDB transaction.	
Dependencies	FR-01, MongoDB transactions, Supabase seedUserPermissions()	
Acceptance Criteria	- All 5 steps navigable; Back button returns without losing data - Min 1 warehouse required; Add/Remove warehouse supported - Min 1 category required; pre-populates based on industry selection - Duplicate admin email returns 409 Conflict - On success: all records created, auth cookie set, redirect to dashboard - Progress bar shows current step (1-5) with numbered circles	

FR-03	Password Reset via Email Token	High
Description	Users request a reset link sent to their registered email. The link contains a 1-hour expiry token. Submitting a new password via the token updates the hashed password and invalidates the token.	
Dependencies	FR-01, GMAIL_USER, GMAIL_APP_PASSWORD, NEXT_PUBLIC_APP_URL	
Acceptance Criteria	- Request endpoint always returns 200 regardless of email existence (security) - Email sent within 30s of request if account exists - Token expires in 1 hour; expired token shows clear error - Used token cannot be reused - New password min 8 characters; confirmation must match	

FR-04	User Invitation System	High
Description	Super admin invites warehouse managers by email. System generates a 3-day expiry token, sends an invitation email with an acceptance link. The invitee fills in their name/phone/password and a User account is created with the specified warehouse assignments. Confirmation email sent to admin.	
Dependencies	FR-01, FR-06, Invitation model, Supabase seedUserPermissions()	
Acceptance Criteria	• Only super_admin can send invitations (403 for other roles) • Invitation email delivered within 60s with inviter name and warehouse list • Token expires after 3 days; expired token shows error on acceptance page • Accepted invitation creates user, populates Supabase permission table • Duplicate pending invitation to same email is blocked • On success: new user auto-logged in and redirected to dashboard	

FR-05	Product Catalogue Management	Critical
Description	Full CRUD for products. Each product has a unique SKU per company, a unit price, unit type, and optional flags for fragility and expiry tracking. Deletion is soft archive (isActive: false) — data preserved. Products can be created with optional initial stock in one atomic transaction via /api/products/create-with-stock.	
Dependencies	FR-02, ProductCategory model, Stock model (create-with-stock endpoint)	
Acceptance Criteria	• SKU must be unique per company; duplicate returns 400 with clear message • List supports search by name/SKU, filter by category, warehouse, and status • Archive requires confirmation modal; shows intent and data preservation note • Archived products excluded from stock entry dropdowns • Product with hasExpiryDate=true shows expiry date field on stock entry • Fragile and Expiry badges shown on product list and detail page	

FR-06	Warehouse Management	Critical
Description	Super admin creates and manages warehouse locations. Each has an address, capacity (units), optional manager array, and contact details. Warehouse managers are restricted to their assigned warehouses. Deletion requires zero active stock.	
Dependencies	FR-02, Supabase permissions, User model	
Acceptance Criteria	• Only super_admin can create, edit, or delete warehouses (403 for others) • Warehouse card shows utilization % with color coding: green <50%, yellow 50-70%, orange 70-90%, red >=90% • Deletion blocked if warehouse has active stock; returns 400 • Manager assignment updates Supabase permission table immediately • Warehouse manager sees only assigned warehouses in all dropdowns	

FR-07	Stock Entry Recording	Critical
-------	-----------------------	----------

Description	Users record stock arriving at a warehouse. Each entry creates a new Stock batch with a unique batchId, records quantity received, entry date, optional photo (Cloudinary), optional expiry date, and optional purchase price. Triggers a warehouse capacity alert asynchronously after entry.
Dependencies	FR-05, FR-06, Stock model, Cloudinary, Alert generator
Acceptance Criteria	• Each submission creates a unique Stock batch with generated batchId • Entry date defaults to today; cannot be set in the future • Expiry date field shown only if product.hasExpiryDate is true • Photo upload: image MIME only, max 5MB, stored on Cloudinary • Warehouse capacity alert triggered if utilization >=90% after entry (non-blocking) • Success toast shown; redirects to stock list on completion

FR-08	Stock Exit Recording	Critical
Description	Users record stock leaving a warehouse by selecting a specific batch and quantity. Records customer name, order reference, optional exit photo. Decrements quantityAvailable on the batch. Triggers a low-stock check alert asynchronously. Cannot exit more than available quantity.	
Dependencies	FR-07, Stock model, StockMovement model, Alert generator	
Acceptance Criteria	• Quantity input capped at quantityAvailable for selected batch • Attempting to exit more than available returns 400 Insufficient Stock • StockMovement record created with movementType=out • Low-stock alert generated if quantity drops below product.reorderLevel • Warehouse manager can only exit from assigned warehouses	

FR-09	Stock Transfer Between Warehouses	High
Description	Move stock from one warehouse to another. Creates a StockMovement with movementType=transfer, decrements source batch, creates destination batch with parentBatchId linking to source. Validates destination capacity. Optional transport details (vehicle number, driver name, expected delivery). Entire operation is atomic via MongoDB transaction.	
Dependencies	FR-07, FR-06, MongoDB transaction, Supabase permission check	
Acceptance Criteria	• Transfer blocked if destination capacity would be exceeded • Transfer blocked if quantity exceeds source batch available quantity • Warehouse manager cannot transfer to/from warehouses they do not manage • Destination batch carries parentBatchId = source batchId for traceability • Transfer appears in movement history for both source and destination	

FR-10	Automated Stock Aging Calculation	Critical
-------	-----------------------------------	----------

Description	Every stock batch tracks age in days from entry date. Age recalculated: (a) on each save via Mongoose pre-save hook, and (b) daily via cron job. Status thresholds: Healthy (0-119 days), At Risk (120-179 days), Dead (180+ days). Cron secured by CRON_SECRET bearer token.
Dependencies	Stock model (pre-save hook + static method), CRON_SECRET, Vercel cron
Acceptance Criteria	• ageInDays = Math.floor((now - entryDate) / 86400000) • Status: healthy <120 days, at_risk 120-179, dead >=180 • Cron endpoint POST /api/cron/update-aging requires Authorization: Bearer CRON_SECRET • Status displayed as color-coded badge: green / orange / red • Cron response includes stats: totalProcessed, updated, alertsGenerated

FR-11	Alert System — All 5 Alert Types	High
Description	Daily cron (GET /api/cron/generate-alerts, 1:00 AM UTC) and mutation triggers generate 5 alert types: (1) dead_inventory (Critical, >=180 days), (2) aging (Warning, 120-179 days), (3) low_stock (Warning, quantity < product.reorderLevel — mutation-triggered after exit), (4) expiry_warning (Critical <7 days, Warning 8-30 days), (5) warehouse_full (Warning >=90%, Critical >=100%). All alerts deduplicated against existing open/acknowledged alerts. Critical alerts trigger email to super_admin.	
Dependencies	FR-10, Alert model, alert-generator.ts, checkLowStockAlert(), checkWarehouseCapacityAlert()	
Acceptance Criteria	• Dead inventory and aging alerts generated by daily cron; low stock and capacity by mutations • No duplicate alert created for same stock + type in open or acknowledged state • Critical alerts trigger email notification to super_admin (non-blocking) • Alert includes title, message, recommendation, and full metadata • Auto-resolution: low_stock resolved when restocked; warehouse_full resolved when capacity frees	

FR-12	Alert Lifecycle Management	High
Description	Users can Acknowledge (records who/when), Resolve (requires optional notes), or Dismiss any open alert. Status transitions: open -> acknowledged -> resolved dismissed. Alerts list sorted: critical -> warning -> info. Default view shows open alerts only; resolved/dismissed filterable.	
Dependencies	FR-11, Alert model, PATCH /api/alerts/[id]	
Acceptance Criteria	• Acknowledge: sets acknowledgedBy, acknowledgedAt, status = acknowledged • Resolve: sets resolvedBy, resolvedAt, resolvedNotes, status = resolved • Dismiss: status = dismissed with no additional data required • Alert detail modal shows message + recommendation + full metadata • Alert list sorted by severity: critical first, then warning, then info	

FR-13	In-App Notifications & Email Alerts	Medium
-------	-------------------------------------	--------

Description	Per-user notification feed for key events: stock_added, stock_low, stock_dead, transfer_initiated, transfer_completed, user_joined, product_added, product_updated. Users can mark individual or all notifications as read, delete notifications. Critical system alerts also send HTML emails via Gmail SMTP.
Dependencies	Notification model, notifications.ts, GMAIL_USER, GMAIL_APP_PASSWORD
Acceptance Criteria	• GET returns paginated notifications with unreadCount • PATCH /[id] marks as read, sets readAt timestamp • POST /mark-all-read marks all unread for current user • Email for critical alerts sent within 60s; send failure is non-blocking • Notifications scoped strictly to current user

FR-14	Aging Stock Report	High
Description	GET /api/reports/aging returns all stock batches grouped by status with summary statistics. Supports filtering by warehouse, category, and status. Accessible to all authenticated roles; warehouse managers see only their assigned warehouses.	
Dependencies	FR-10, Stock model, Product model	
Acceptance Criteria	• Summary: totalItems, totalValue, healthy/atRisk/dead each with count, value, and percentage • Supports query params: ?warehouse=id, ?category=id, ?status=at_risk • Warehouse manager report scoped to assigned warehouses only • Report UI shows three sections with stock list and summary card for each status	

FR-15	Inventory Dashboard	Critical
Description	Main dashboard aggregates: health score (0-100), total products, total inventory value, dead stock count, at-risk count, warehouse utilization, recent activity timeline (last 10 movements), and action items (dead/at-risk batches). Data fetched server-side with parallel queries. Empty state shown for new accounts.	
Dependencies	FR-05 through FR-14, dashboard-data.ts, GET /api/dashboard	
Acceptance Criteria	• Health score = 100 - (deadCount x 15 + atRiskCount x 5), minimum 10 • Empty state when totalProducts = 0 with CTA to add first product • Value formatted: Rs.1K / Rs.1L / Rs.1Cr based on magnitude • Utilization bar: green <50%, yellow 50-70%, orange 70-90%, red >=90% • Warehouse manager dashboard scoped to assigned warehouses only	

FR-16	Photo Upload for Stock Verification	High
Description	Stock entry and exit forms support photo upload. Photos uploaded to Cloudinary via POST /api/upload with multipart/form-data. Returned URL stored in stock/movement record. Displayed as thumbnails in stock list; click opens full-screen modal.	
Dependencies	CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, cloudinary.ts	
Acceptance Criteria	• Accepted MIME types: JPEG, PNG, WebP only; max 5MB • Upload UI: dashed dropzone with drag-and-drop; preview on selection • Returns Cloudinary secure_url and publicId on success • Auth required for upload; unauthorized requests rejected with 401	

FR-17	Role-Based Access Control	Critical
Description	Two roles: super_admin (full access) and warehouse_manager (scoped to assigned warehouses). Access enforced at three layers: (1) Next.js edge middleware, (2) API route handlers, (3) Supabase PostgreSQL permission table. Role injected into request headers by middleware.	
Dependencies	middleware.ts, rbac.ts, pg-permissions.ts, Supabase user_warehouse_permissions table	
Acceptance Criteria	• Middleware injects x-user-id, x-user-role, x-company-id headers on every request • Warehouse manager accessing unassigned warehouse data returns 403 • Delete product, create/edit/delete warehouse: super_admin only • Unauthorized page access redirects to /dashboard?error=unauthorized • Supabase permission table is the enforcement source of truth for warehouse access	

Non-Functional Requirements

Quality attributes the system must satisfy independent of features

Performance Targets

Metric	Target	Measurement
Dashboard first load (TTFB)	< 2s on cold Vercel serverless start	Vercel Analytics / Lighthouse
API p95 response time	< 500ms for all endpoints	Vercel function execution logs
LCP (Largest Contentful Paint)	< 2.5s on 4G mobile	Lighthouse / Core Web Vitals
CLS (Cumulative Layout Shift)	< 0.1	Chrome DevTools / Web Vitals
Categories endpoint (cached)	< 50ms (1-hour Next.js cache)	Response time header
Warehouse names endpoint (cached)	< 50ms (5-min cache TTL)	Response time header
Cron job execution (all companies)	< 60 seconds total	Vercel function timeout log
Image upload to Cloudinary	< 3s for files up to 5MB	API response time measurement

Security Requirements

Area	Requirement
Password storage	bcryptjs hash, salt rounds ≥ 10 ; plaintext never stored or logged
Token storage	JWT in HttpOnly, Secure, SameSite=Lax cookie; never accessible from JavaScript
CSRF protection	SameSite cookie policy; no CSRF token needed for same-site requests
Route protection	All routes verified by Next.js edge middleware; no bypass possible
Cron security	All /api/cron/* endpoints require Authorization: Bearer CRON_SECRET
File validation	MIME type and size (5MB max) validated before Cloudinary upload
Token expiry	Reset tokens: 1 hour; invitation tokens: 3 days; both single-use
Secrets	All secrets via environment variables only; never hardcoded in source
Multi-tenancy	Every DB query includes companyId filter; cross-tenant access impossible

Scalability & Availability

- Availability target: 99.5% monthly uptime (excluding planned Vercel maintenance windows)
- All collection queries indexed by companyId — no full-collection scans in production
- Stateless serverless functions — horizontal scaling is automatic on Vercel
- MongoDB Atlas M0/M10 with built-in replication; no single point of failure

- Stock table designed to handle 100,000+ batch records per company with compound indexes
- Alert and notification creation always non-blocking — stock operations succeed even if alert fails

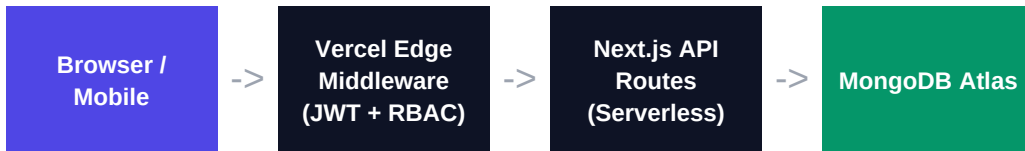
Accessibility & Compatibility

- Target: WCAG 2.1 Level AA compliance
- Browsers: Chrome 100+, Firefox 100+, Safari 15+, Edge 100+; no IE support
- Screen sizes: 375px (iPhone SE) through 2560px (4K desktop)
- All interactive elements keyboard-accessible (Tab/Enter/Escape); visible focus indicators
- Color never sole conveyor of state — always paired with text label or icon
- Dark mode supported via next-themes; preference persisted to localStorage

System Architecture & Technology Stack

Architecture Overview

Inventory Tracker is a monolithic Next.js application deployed as serverless functions on Vercel. The architecture is request-response with no persistent server process. Edge middleware handles authentication and RBAC on every request before it reaches any route handler. The primary data store is MongoDB Atlas (via Mongoose). A secondary Supabase PostgreSQL table enforces warehouse-level permissions as the source of truth. Images are stored on Cloudinary CDN. Cron jobs are scheduled via vercel.json.



Also: Supabase PostgreSQL (permission table) | Cloudinary (image CDN) | Vercel Cron (daily jobs) | Gmail SMTP (email)

Technology Stack

Category	Technology	Version	Purpose
Framework	Next.js	16.1.1	Full-stack React framework with App Router, RSC, API routes
UI Library	React	19.2.3	Latest React with Server Components and concurrent features
Language	TypeScript	5.x	Strict mode across all frontend and backend code
Database (Primary)	MongoDB Atlas	Mongoose 9.0.2	Document store via Mongoose ODM; all business data
Database (Secondary)	Supabase PostgreSQL	@supabase/supabase-js 2.x	user_warehouse_permissions table; access enforcement source of truth
Auth	jsonwebtoken + jose	9.0.3 + 6.1.3	JWT signing (Node) and verification (Edge runtime)
Password	bcryptjs	3.0.3	Password hashing with configurable salt rounds
Styling	TailwindCSS	v4	Utility-first CSS with PostCSS v4 integration
UI Components	Radix UI + shadcn	Various	Headless accessible primitives: Label, Progress, Slot
Icons	lucide-react + react-icons	0.562.0 + 5.5.0	SVG icon libraries for UI elements
Forms	react-hook-form + Zod	7.69.0 + 4.2.1	Form state management with runtime schema validation

Category	Technology	Version	Purpose
Toast Notifications	sonner	2.0.7	Toast notification system for user feedback
Email	Nodemailer	7.0.13	SMTP email via Gmail App Password
Image Storage	Cloudinary	2.9.0	Cloud image upload and CDN delivery
Themes	next-themes	0.4.6	Dark/light mode management with localStorage persistence
Analytics	@vercel/analytics	1.6.1	Page views and Core Web Vitals tracking
Deployment	Vercel	Managed	Serverless functions, edge middleware, cron scheduling
CI	ESLint	9 + eslint-config-next	Lint enforcement on every commit

Project Folder Structure

Complete directory layout with purpose of every folder:

Path	Purpose
my-app/	Root application directory
app/	Next.js App Router — all pages and API routes
app/api/	API route handlers (41 endpoints across 13 groups)
app/api/auth/	Login, logout, me, forgot-password, reset-password
app/api/products/	Product CRUD + create-with-stock endpoint
app/api/warehouses/	Warehouse CRUD with capacity and manager management
app/api/stock/	Stock entry, exit, and transfer endpoints
app/api/alerts/	Alert listing, detail, and status update (PATCH)
app/api/notifications/	User notifications: list, mark read, delete, mark-all-read
app/api/reports/	Aging stock report endpoint
app/api/dashboard/	Aggregated dashboard KPI data
app/api/users/	User listing + invitation send and accept flow
app/api/cron/	update-aging (POST) and generate-alerts (GET) — secured by CRON_SECRET
app/api/upload/	Cloudinary image upload endpoint
app/api/onboarding/	Company + admin creation endpoint
app/api/categories/	Product categories list (cached 1 hour)
app/api/health/	Public health check endpoint
app/dashboard/	All protected dashboard pages (sidebar layout)
app/dashboard/inventory/	Products list, detail, edit pages
app/dashboard/warehouses/	Warehouses list, detail, new, edit pages
app/dashboard/stock/	Stock list, entry, exit, transfer pages
app/dashboard/alerts/	Alerts list with filter and action modals
app/dashboard/reports/	Reports navigation and aging report page
app/login/	Public login page
app/onboarding/	Public 5-step onboarding wizard page
app/forgot-password/	Password reset request page
app/reset-password/[token]/	Password reset form (token from email)
app/invite/accept/	Invitation acceptance page (warehouse managers)

Path	Purpose
lib/	Shared utility and business logic library (16 files)
lib/auth.ts	JWT sign, verify, cookie set/remove, getCurrentUser()
lib/db.ts	MongoDB connection with caching (prevents hot-reload reconnects)
lib/email.ts	Nodemailer + Gmail SMTP transporter setup
lib/alert-generator.ts	Alert generation and auto-resolution logic
lib/dashboard-data.ts	Dashboard KPI aggregation with parallel queries
lib/pg-permissions.ts	Supabase PostgreSQL permission CRUD functions
lib/rbac.ts	Role permission constants and canDo() helper functions
lib/cached-data.ts	Next.js unstable_cache for categories and warehouse names
lib/cloudinary.ts	Cloudinary v2 SDK initialization
lib/api-response.ts	Shared jsonError(), jsonSuccess(), CommonErrors formatters
lib/env-validation.ts	Startup env var validation — throws on missing required vars
models/	Mongoose schema definitions (11 models)
components/	React components: ui/ (shadcn), layout/ (sidebar), onboarding/
middleware.ts	Next.js edge middleware: JWT verification + RBAC + header injection
vercel.json	Cron schedule: generate-alerts at 01:00 UTC daily
next.config.ts	serverExternalPackages: [mongoose] for edge compatibility
.env / .env.example	Environment variables (development) and template

Database Schema — Data Models

All 11 Mongoose models with fields, types, constraints, and relationships

All models use timestamps: true (auto-managed createdAt/updatedAt). Multi-tenancy is enforced by companyId (ref: SystemConfig) on every core collection. Required fields are marked with * in the Req column.

User

models/User.ts

Manages users within the system. Supports two roles: super_admin and warehouse_manager.

Field	Type	Req	Default	Notes
name	String	*	—	Full name
email	String	*	—	Unique globally; used for login
password	String	*	—	bcrypt hash
role	String enum	*	—	super_admin warehouse_manager
phone	String		—	Optional contact number
companyId	ObjectId		—	ref: SystemConfig
assignedWarehouses	ObjectId[]		[]	ref: Warehouse — display only
invitedBy	ObjectId		—	ref: User (self-reference)
status	String enum	*	active	active inactive pending
isActive	Boolean	*	true	Active/inactive flag
emailVerified	Boolean	*	false	Email verification status
invitationToken	String		—	For signup link
invitationExpiry	Date		—	Invitation token expiration
resetToken	String		—	Password reset token (hashed)
resetTokenExpiry	Date		—	Reset token expiration
preferences.emailNotifications	Boolean	*	true	Email notification opt-in
preferences.dailyDigest	Boolean	*	true	Daily digest opt-in
preferences.criticalAlertsOnly	Boolean	*	false	Only critical alerts
lastLogin	Date		—	Last login timestamp
Indexes	email (unique) { companyId: 1 }			
References	companyId -> SystemConfig assignedWarehouses[] -> Warehouse invitedBy -> User			

SystemConfig

models/SystemConfig.ts

Company-level configuration. One document per company. Acts as the company root entity.

Field	Type	Req	Default	Notes
companyName	String	*	—	Company display name
industryType	String	*	—	Industry classification
currency	String	*	USD	Currency code (e.g., INR, USD)
concerns	String[]		[]	Selected aging concerns
agingThresholds.atRisk	Number		60	Days threshold for at_risk (display only)
agingThresholds.dead	Number		90	Days threshold for dead (display only)
alertSettings.emailEnabled	Boolean		true	Enable email alerts
alertSettings.smsEnabled	Boolean		false	Enable SMS alerts (not implemented)
Indexes	timestamps auto-indexed			
References	Referenced by all core models via companyId			

Warehouse

[models/Warehouse.ts](#)

Physical warehouse locations. Supports multiple managers and capacity tracking.

Field	Type	Req	Default	Notes
companyId	ObjectId	*	—	ref: SystemConfig
name	String	*	—	Warehouse name (trimmed)
warehouseCode	String	*	—	Unique code (uppercase); globally unique
address.street	String	*	—	Street address
address.city	String	*	—	City
address.state	String	*	—	State/Province
address.pin	String	*	—	Postal code
address.country	String	*	India	Country
managers	ObjectId[]		[]	ref: User — array of manager IDs
capacity	Number	*	1000	Total capacity in units; min 1
contactPhone	String		""	Warehouse contact phone
contactEmail	String		""	Warehouse contact email
notes	String		""	Additional notes
isActive	Boolean	*	true	Soft delete flag
Indexes	{companyId:1, isActive:1} {managers:1} {companyId:1, name:1} partial (isActive only) warehouseCode (globally unique)			

References	companyId -> SystemConfig managers[] -> User
------------	--

ProductCategory

models/ProductCategory.ts

Product categories with aging concern levels. Determines how aging thresholds apply.

Field	Type	Req	Default	Notes
companyId	ObjectId	*	—	ref: SystemConfig
name	String	*	—	Category name
agingConcern	String enum	*	—	slow moderate fast expiry
agingDays	Number		—	Aging threshold days (180/90/60 or null for expiry)
isFragile	Boolean	*	false	Products in this category are fragile
requiresPhotoVerification	Boolean	*	false	Photo required on entry/exit
isActive	Boolean	*	true	Soft delete flag
Indexes	{companyId:1} {name:1} {companyId:1, name:1} unique			
References	companyId -> SystemConfig Referenced by: Product.category			

Product

models/Product.ts

Product master data with pricing, unit type, and feature flags.

Field	Type	Req	Default	Notes
companyId	ObjectId	*	—	ref: SystemConfig; indexed
sku	String	*	—	Stock Keeping Unit; unique per company
name	String	*	—	Product name; text-indexed for search
category	ObjectId	*	—	ref: ProductCategory
subCategory	String		—	Optional subcategory string
unitPrice	Number	*	—	Price per unit
unitType	String enum	*	piece	piece kilogram liter box meter
isFragile	Boolean	*	false	Fragile product flag
hasExpiryDate	Boolean	*	false	Track expiry date on stock entries
reorderLevel	Number	*	10	Trigger low-stock alert below this quantity
images	String[]		[]	Array of image URLs
isActive	Boolean	*	true	Soft delete flag
Indexes	companyId name (text index for search) {companyId:1, sku:1} unique			
References	companyId -> SystemConfig category -> ProductCategory Referenced by: Stock, StockMovement, Alert			

Stock

models/Stock.ts

Inventory batches at warehouse level. Core entity for all stock tracking. Each entry/transfer creates a batch.

Field	Type	Req	Default	Notes
companyId	ObjectId	*	—	ref: SystemConfig
productId	ObjectId	*	—	ref: Product
warehouseId	ObjectId	*	—	ref: Warehouse
createdBy	ObjectId	*	—	ref: User — who created this batch
batchId	String	*	—	Unique batch identifier
parentBatchId	String		null	Source batchId if transferred; null for direct entries
quantityReceived	Number	*	—	Initial quantity at entry; min 0
quantityAvailable	Number	*	—	Current available quantity; min 0
quantityDamaged	Number	*	0	Damaged quantity; min 0
entryDate	Date	*	now	When stock was received
expiryDate	Date		null	Product expiry date if hasExpiryDate
ageInDays	Number	*	0	Calculated: (now - entryDate) / 86400000; pre-save hook
status	String enum	*	healthy	healthy at_risk dead
entryPhotos	Array		[]	Array of {url, uploadedBy (ref:User), timestamp}
Indexes	{companyId:1} {companyId:1, warehouseId:1} {companyId:1, status:1} {ageInDays:-1} {warehouseId:1, status:1} {parentBatchId:1} {productId:1, warehouseId:1} {companyId:1, batchId:1, warehouseId:1} unique			
References	companyId -> SystemConfig productId -> Product warehouseId -> Warehouse createdBy -> User			

StockMovement

models/StockMovement.ts

Audit trail for every stock transaction. Records in, out, transfer, damage, and adjustment events.

Field	Type	Req	Default	Notes
stockId	ObjectId	*	—	ref: Stock — batch being moved
productId	ObjectId	*	—	ref: Product
warehouseId	String	*	—	Warehouse ID (typed as String, refs Warehouse)
performedBy	ObjectId	*	—	ref: User — who performed movement
movementType	String enum	*	—	in out transfer damage adjustment
quantity	Number	*	—	Quantity moved; min 1
unitPrice	Number		—	Unit price at time of movement
totalValue	Number		—	quantity x unitPrice
reason	String	*	—	Reason for movement (trimmed)
photos	Array		[]	Array of {url, type: entry exit damage}
timestamp	Date	*	now	Movement timestamp
fromWarehouse	String		—	Transfer: source warehouse name/ID
fromStockId	ObjectId		—	ref: Stock — source batch for transfer
toWarehouse	String		—	Transfer: destination warehouse name/ID
toStockId	ObjectId		—	ref: Stock — destination batch for transfer
transportDetails	Object		—	{vehicleNumber, driverName, driverPhone, expectedDelivery}
transferStatus	String enum		—	initiated in_transit completed cancelled
Indexes	{stockId:1} {timestamp:-1} {movementType:1} {transferStatus:1}			
References	stockId -> Stock fromStockId -> Stock toStockId -> Stock productId -> Product performedBy -> User initiatedBy -> User completedBy -> User			

Alert

models/Alert.ts

System-generated alerts for inventory issues. Supports full lifecycle: open -> acknowledged -> resolved | dismissed.

Field	Type	Req	Default	Notes
companyId	ObjectId	*	—	ref: SystemConfig
stockId	ObjectId		—	ref: Stock — related batch if applicable

Field	Type	Req	Default	Notes
productId	ObjectId		—	ref: Product
warehouseId	ObjectId	*	—	ref: Warehouse — affected location
type	String enum	*	—	dead_inventory aging low_stock expiry_warning warehouse_full
severity	String enum	*	—	critical warning info
title	String	*	—	Alert headline
message	String	*	—	Full alert description
recommendation	String	*	—	Recommended action for user
status	String enum	*	open	open acknowledged resolved dismissed
acknowledgedBy	ObjectId		—	ref: User
acknowledgedAt	Date		—	Acknowledgment timestamp
resolvedBy	ObjectId		—	ref: User
resolvedAt	Date		—	Resolution timestamp
resolvedNotes	String		—	Notes on resolution
metadata	Mixed		—	{ageInDays, quantity, value, expiryDate, capacityPercent}
Indexes	{companyId:1, status:1} {stockId:1} {type:1, severity:1} {createdAt:-1}			
References	companyId -> SystemConfig stockId -> Stock productId -> Product warehouseId -> Warehouse acknowledgedBy -> User resolvedBy -> User			

Notification

models/Notification.ts

Per-user notification feed. Created for stock events, alerts, and user activity.

Field	Type	Req	Default	Notes
userId	ObjectId	*	—	ref: User — recipient; indexed
type	String enum	*	—	stock_added stock_low stock_dead transfer_initiated transfer_completed user_joined damage_reported product_added product_updated
title	String	*	—	Notification headline
message	String	*	—	Full notification text
relatedEntity	Mixed		—	{type: string, id: ObjectId} — entity reference
actionUrl	String		—	Deep link (e.g., /stock/123)
status	String enum	*	unread	unread read
priority	String enum	*	medium	low medium high critical
readAt	Date		—	When notification was read
Indexes	{userId:1} {type:1} {status:1} {priority:1} {userId:1, status:1, createdAt:-1} {userId:1, priority:1, createdAt:-1}			
References	userId -> User			

Invitation

models/Invitation.ts

User invitation workflow: invite -> accept -> user created. Token-based, 3-day expiry.

Field	Type	Req	Default	Notes
email	String	*	—	Invited email address; indexed
role	String enum	*	—	super_admin warehouse_manager
assignedWarehouses	ObjectId[]		[]	ref: Warehouse — assigned on acceptance
token	String	*	—	Unique invitation token; globally unique
invitedBy	ObjectId	*	—	ref: User — who sent invitation
companyId	ObjectId	*	—	ref: SystemConfig
status	String enum	*	pending	pending accepted expired
sentAt	Date	*	now	Invitation sent timestamp
expiresAt	Date	*	—	Expiration date (+3 days from sentAt)
acceptedAt	Date		—	Acceptance timestamp

Field	Type	Req	Default	Notes
<code>personalMessage</code>	String		—	Custom message from inviter
Indexes	{email:1} {token:1} unique {email:1, status:1} {expiresAt:1, status:1}			
References	invitedBy -> User companyId -> SystemConfig assignedWarehouses[] -> Warehouse			

AuditLog

models/AuditLog.ts

Compliance audit trail for all entity mutations. Not exposed via API in v1.0.

Field	Type	Req	Default	Notes
<code>action</code>	String	*	—	Action name (e.g., create, update, delete); indexed
<code>userId</code>	ObjectId		—	ref: User — who performed action
<code>entityType</code>	String	*	—	Entity type (e.g., Stock, Product); indexed
<code>entityId</code>	ObjectId	*	—	ID of the changed entity
<code>metadata</code>	Mixed		—	Change details: before/after values
<code>timestamp</code>	Date	*	now	Action timestamp; indexed
<code>ipAddress</code>	String		—	Client IP address
<code>userAgent</code>	String		—	Browser user agent string
Indexes	{action:1} {entityType:1} {timestamp:1} {entityType:1, entityId:1, timestamp:-1} {userId:1, timestamp:-1}			
References	userId -> User			

API Specification

All 41 endpoints — method, path, auth requirement, role restriction, and description

Base URL: configured via NEXT_PUBLIC_APP_URL. All authenticated endpoints read the auth-token HttpOnly cookie. Role column shows minimum required role. All responses include a success boolean field.

Authentication

Method	Path	Auth	Role	Description
POST	/api/auth/login	No	—	Login with email + password; sets auth-token cookie (7 days)
POST	/api/auth/logout	Yes	Any	Clear auth-token cookie
GET	/api/auth/me	Yes	Any	Get current user: id, name, email, role, assignedWarehouses, preferences
POST	/api/auth/forgot-password	No	—	Request password reset email; always returns 200 for security
GET	/api/auth/reset-password	No	—	Verify reset token validity; returns valid:bool, email
POST	/api/auth/reset-password	No	—	Set new password using token from email; invalidates token
POST	/api/auth/signup	No	—	Disabled — returns 410 Gone; use /onboarding instead

Onboarding

Method	Path	Auth	Role	Description
POST	/api/onboarding	Optional	—	Create company + warehouses + categories + admin in one atomic operation

Products

Method	Path	Auth	Role	Description
GET	/api/products	Yes	Any	List products; filter by search, category, warehouse, status
POST	/api/products	Yes	Any	Create new product
GET	/api/products/[id]	Yes	Any	Get single product details

Method	Path	Auth	Role	Description
PUT	/api/products/[id]	Yes	Any	Update product fields
DELETE	/api/products/[id]	Yes	Super Admin	Soft-archive product (isActive=false)
GET	/api/products/[id]/details	No	—	Product with stock info (public endpoint)
POST	/api/products/create-with-stock	Yes	Any	Create product + initial stock batch atomically

Warehouses

Method	Path	Auth	Role	Description
GET	/api/warehouses	Yes	Any	List warehouses with metrics; ?minimal=true for dropdown list
POST	/api/warehouses	Yes	Super Admin	Create new warehouse
GET	/api/warehouses/[id]	Yes	Any	Get single warehouse details
PUT	/api/warehouses/[id]	Yes	Super Admin	Update warehouse (name, address, managers, capacity)
DELETE	/api/warehouses/[id]	Yes	Super Admin	Soft-archive warehouse (must have zero active stock)

Stock

Method	Path	Auth	Role	Description
POST	/api/stock/entry	Yes	Any	Record stock entry; creates batch, triggers capacity alert
GET	/api/stock/entry	Yes	Any	List stock entries; filter by product, warehouse
POST	/api/stock/exit	Yes	Any	Record stock exit; decrements batch, triggers low-stock alert
GET	/api/stock/exit	Yes	Any	List stock movements (exits); filter by product, warehouse, type
POST	/api/stock/transfer	Yes	Any	Transfer stock between warehouses (atomic); validates capacity
GET	/api/stock/transfer	Yes	Any	List transfer history (last 50)

Alerts

Method	Path	Auth	Role	Description
GET	/api/alerts	Yes	Any	List alerts; filter by status, type, severity, warehouseId
GET	/api/alerts/[id]	Yes	Any	Get single alert with populated references
PATCH	/api/alerts/[id]	Yes	Any	Update alert status: action = acknowledge resolve dismiss

Notifications

Method	Path	Auth	Role	Description
GET	/api/notifications	Yes	Any	List user notifications; paginated; filter by status, priority
PATCH	/api/notifications/[id]	Yes	Any	Mark single notification as read
DELETE	/api/notifications/[id]	Yes	Any	Delete notification permanently
POST	/api/notifications/mark-all-read	Yes	Any	Mark all user notifications as read

Users & Invitations

Method	Path	Auth	Role	Description
GET	/api/users	Yes	Any	List company users; ?role=filter
POST	/api/users/invite	Yes	Super Admin	Send warehouse manager invitation email (3-day token)
GET	/api/users/invite/accept	No	—	Verify invitation token; returns invitation details
POST	/api/users/invite/accept	No	—	Accept invitation; creates user account + Supabase permissions

Reports, Dashboard & System

Method	Path	Auth	Role	Description
GET	/api/reports/aging	Yes	Any	Aging report: healthy/at-risk/dead breakdown with totals
GET	/api/dashboard	Yes	Any	Aggregated dashboard KPIs: health score, metrics, recent activity

Method	Path	Auth	Role	Description
GET	/api/categories	Yes	Any	List product categories (cached 1 hour)
POST	/api/upload	Yes	Any	Upload image to Cloudinary; returns secure_url + publicId
GET	/api/analytics	No	—	Vercel Analytics proxy (last 7 days): country, browser, OS data
GET	/api/health	No	—	Health check: DB status, response time, version, environment
POST	/api/cron/update-aging	Bearer	CRON_SECRET	Recalculate ageInDays + status for all stock batches
GET	/api/cron/generate-alerts	Bearer	CRON_SECRET	Generate alerts for all companies (runs daily at 01:00 UTC)

Authentication & Authorization

JWT cookie flow, middleware, dual permission system, and role matrix

Authentication Flow

Step	Action
1	User submits email + password to POST /api/auth/login
2	API queries MongoDB for User by email; verifies bcrypt hash
3	On success: signToken({userId, email, role, companyId}) -> JWT (7-day expiry)
4	JWT stored as HttpOnly, Secure, SameSite=Lax cookie named auth-token
5	Every subsequent request: Next.js edge middleware reads auth-token cookie
6	Middleware calls verifyTokenEdge() (jose library, edge-compatible)
7	Valid token: middleware injects x-user-id, x-user-role, x-company-id headers
8	API route handlers read these headers; never re-verify the token independently
9	Invalid/expired token: clear cookie -> redirect to /login (pages) or 401 (API)
10	Logout: POST /api/auth/logout clears auth-token cookie via removeAuthCookie()

Dual Permission System

Warehouse-level access is enforced by TWO independent systems. MongoDB stores assignedWarehouses on the User document for display purposes. Supabase PostgreSQL table user_warehouse_permissions is the enforcement source of truth — checked on every warehouse-scoped API operation via pg-permissions.ts functions.

Function (lib/pg-permissions.ts)	Purpose
hasWarehouseAccess(userId, warehouseId)	Check if user can access a specific warehouse
getUserWarehouseIds(userId)	Get all warehouse IDs accessible to a user
seedUserPermissions(userId, warehouseIds, companyId)	Bulk insert on onboarding/invite accept
grantWarehouseAccess(userId, warehouseId)	Grant single warehouse access
revokeWarehouseAccess(userId, warehouseId)	Revoke single warehouse access
revokeAllUserPermissions(userId)	Full cleanup on user deletion
revokeAllWarehousePermissions(warehouseId)	Full cleanup on warehouse deletion

Authorization Matrix

Feature / Action	Super Admin	Warehouse Manager
Login / Logout / Forgot Password	Yes	Yes
View Dashboard	Yes (all warehouses)	Yes (assigned only)
View Products List	Yes	Yes
Create / Edit Product	Yes	Yes
Archive Product	Yes	No
Create Product with Stock	Yes	Yes (assigned warehouses)
View Warehouses	Yes (all)	Yes (assigned only)
Create / Edit / Delete Warehouse	Yes	No
Record Stock Entry	Yes	Yes (assigned warehouses)
Record Stock Exit	Yes	Yes (assigned warehouses)
Transfer Stock	Yes (any)	Yes (assigned warehouses only)
View All Alerts	Yes	Yes (assigned warehouses)
Acknowledge / Resolve Alerts	Yes	Yes
View Notifications	Yes	Yes (own only)
Invite Users	Yes	No
View Aging Report	Yes (all)	Yes (assigned warehouses)
Upload Photos	Yes	Yes
Access Cron Endpoints	CRON_SECRET bearer token only	No

Section 11 UI / UX Overview

Page map, design system tokens, and key interaction patterns

Page Map

Type	Path	Description
Public	/	Landing page: marketing hero, 5 feature sections, how-it-works, CTA
Public	/login	Email/password login with demo credentials prefill button
Public	/onboarding	5-step company setup wizard (SetupWizard component)
Public	/forgot-password	Password reset request form
Public	/reset-password/[token]	New password form (validates token first)
Public	/invite/accept	Invitation acceptance form (reads token from query param)
Protected	/dashboard	Main dashboard: health score, stats, warehouse snapshot, activity
Protected	/dashboard/inventory	Products list with search, filters, archive action
Protected	/dashboard/inventory/[id]	Product detail: stock by warehouse, movement history
Protected	/dashboard/inventory/[id]/edit	Edit product form (pre-filled)
Protected	/dashboard/products/new	2-step add product: details then optional initial stock
Protected	/dashboard/products/success	Post-creation success page with next steps
Protected	/dashboard/warehouses	Warehouse cards with utilization bars and metrics
Protected	/dashboard/warehouses/new	Create warehouse form
Protected	/dashboard/warehouses/[id]	Warehouse detail: metrics, managers, recent movements
Protected	/dashboard/warehouses/[id]/edit	Edit warehouse (name, address, managers, capacity)
Protected	/dashboard/stock	Stock batches list with aging status and entry photo thumbnails
Protected	/dashboard/stock/entry	Record stock entry form (product, warehouse, qty, date, photo)
Protected	/dashboard/stock/exit	Record stock exit form (batch select, qty, customer, photo)

Type	Path	Description
Protected	/dashboard/stock/transfer	Transfer stock form (source, destination, qty, transport)
Protected	/dashboard/alerts	Alerts list: 3-column severity summary + filtered list + detail modal
Protected	/dashboard/reports	Report navigation cards (aging implemented; others placeholder)

Design System Tokens

Token	Value	Usage
Primary Brand	Indigo (#4f46e5)	Primary buttons, active states, links, section badges
Brand Dark	Navy (#0e1325)	Sidebar background, doc headers, dark surfaces
Success / Healthy	Emerald (#059669)	Healthy stock status, success toasts, positive states
Warning / At Risk	Amber (#d97706)	At-risk stock, warning alerts, orange utilization bar
Critical / Dead	Red (#dc2626)	Dead stock, critical alerts, destructive actions
Info	Blue (#2563eb)	Info callouts, info alerts, secondary info
Background	Frost (#f9fafb)	Page background, table alternate rows, input backgrounds
Text Primary	Ink (#111827)	Headings, labels, important content
Text Secondary	Slate (#4b5563)	Body text, descriptions, secondary information
Text Muted	Muted (#9ca3af)	Captions, table headers, placeholders
Border	Wire (#e5e7eb)	All borders, dividers, card edges
Font — Display	System UI stack	All headings: weight 700-800, tracking -0.02em
Font — Body	System UI stack	Body text: 14px, line-height 1.7, max-width 68ch
Font — Code / Data	ui-monospace / Consolas	Endpoints, IDs, batch codes, env vars
Border Radius	7-8px cards, 20px pills, 3px inline badges	Consistent across all components
Primary Button	Gradient: from-purple-600 to-blue-600	All primary submit/action buttons
Sidebar	Navy background, indigo accent, white text	Fixed 252px sidebar with scroll-spy

Key UX Patterns

Pattern	Behaviour
Empty States	Large icon + heading + description + CTA button; shown when list is empty (e.g., no products, no alerts)
Loading States	Skeleton placeholder from loading.tsx for dashboard; spinner on submit buttons; button text changes to 'Saving...'
Error States	Toast notifications (Sonner) for API errors; red alert banner inside forms for validation; red input borders
Success States	Green toast notifications; success page for product creation with 5-second auto-redirect
Destructive Actions	Always require confirmation modal (archive product, delete notification) before executing
Dark Mode	Toggled by Sun/Moon icon in sidebar; persisted to localStorage; full dark: CSS class-based via next-themes
Photo Preview	Dashed upload dropzone; thumbnail preview on selection; full-screen modal on click in stock list
Inventory Value	Formatted by magnitude: Rs.1K / Rs.1L / Rs.1Cr — not raw numbers
Role Filtering	Warehouse dropdowns silently filtered by assigned warehouses; no error shown — just fewer options
Responsive Nav	Fixed sidebar on desktop (252px); hamburger drawer on mobile; overlay backdrop on mobile open

Security, Performance & Environment Variables

Security Controls

Control	Implementation
XSS Prevention	HttpOnly cookies prevent JS from accessing auth tokens; React auto-escapes JSX output
CSRF Prevention	SameSite=Lax cookie policy; same-origin requests only for state-changing endpoints
SQL Injection	No SQL — MongoDB with Mongoose; parameterized queries via ODM schema enforcement
Secrets Management	All secrets via env vars; env-validation.ts throws on startup if required vars missing
Rate Limiting	Not implemented in v1.0 — planned for v2; Vercel platform-level DDoS protection active
Password Policy	Minimum 8 characters enforced client-side and server-side; bcrypt hash with salt>=10
Audit Logs	AuditLog collection records all mutations with user ID, IP address, and user-agent
Token Security	All tokens (reset, invitation) are single-use, time-limited, and cryptographically random
File Validation	Upload endpoint validates MIME type and enforces 5MB max before Cloudinary upload
Multi-tenancy	All queries include companyId; no join possible across companies at the query level

Environment Variables

Variable	Requirement	Description
MONGODB_URI	Required	MongoDB Atlas connection string
JWT_SECRET	Required	JWT signing key; minimum 32 characters recommended
JWT_EXPIRES_IN	Optional	Token expiry (default: 7d)
EMAIL_USER	Required	Gmail address for sending system emails
EMAIL_APP_PASSWORD	Required	16-character Gmail app-specific password
CRON_SECRET	Required (Prod)	Bearer token securing all /api/cron/* endpoints
NEXT_PUBLIC_APP_URL	Required	Public app URL (e.g., https://app.domain.com)
NEXT_PUBLIC_SUPABASE_URL	Required	Supabase project URL (public)

Variable	Requirement	Description
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Required	Supabase server-side admin key for permissions
<code>CLOUDINARY_CLOUD_NAME</code>	Required	Cloudinary cloud name for image uploads
<code>CLOUDINARY_API_KEY</code>	Required	Cloudinary API key
<code>CLOUDINARY_API_SECRET</code>	Required	Cloudinary API secret
<code>VERCEL_ACCESS_TOKEN</code>	Optional	For analytics proxy endpoint
<code>VERCEL_PROJECT_ID</code>	Optional	For analytics proxy endpoint
<code>VERCEL_TEAM_ID</code>	Optional	For team Vercel deployments
<code>NODE_ENV</code>	Auto	development or production (set by Vercel automatically)

Testing Strategy & Deployment

Testing Strategy

Test Type	Priority	Scope & Approach
Unit Tests	Medium	Test pure utility functions in lib/ (auth.ts signToken/verifyToken, api-response.ts formatters, rbac.ts permission checks). Mock MongoDB and external services.
Integration Tests	High	Test API routes end-to-end against a real MongoDB test database. Cover: login flow, stock entry/exit, transfer atomicity, alert generation, permission enforcement.
E2E Tests	High	Playwright tests for critical user flows: onboarding wizard, login -> dashboard, add product + stock, record exit, acknowledge alert. Run on every PR against staging.
Regression Tests	Medium	Re-run full E2E suite before every production release. Focus on: auth flows, stock operations, alert lifecycle.
Security Tests	High	Test: cross-company data access (should 403), warehouse manager accessing unassigned warehouse (should 403), expired tokens (should 401), cron without bearer token (should 401).
Load Tests	Low (v1.0)	Baseline: simulate 50 concurrent users recording stock entries. Target: all responses < 500ms under load. Tool: k6 or Artillery.
Accessibility Tests	Medium	Automated: axe-core on all pages. Manual: keyboard navigation through complete user flows; screen reader test on Chrome with ChromeVox.

Deployment Strategy

Inventory Tracker is deployed exclusively on Vercel. There are three environments: development (local), staging (Vercel preview deployments on PR), and production (Vercel main branch deployment). All environments use separate MongoDB Atlas clusters and separate Supabase projects.

Environment	Details
Development	Local Next.js dev server (npm run dev); local .env file; MongoDB Atlas shared dev cluster; hot reload active
Staging	Vercel Preview Deployment triggered on every PR; separate env vars; used for E2E and manual QA
Production	Vercel main branch auto-deploy on merge to main; MongoDB Atlas M10+; all env vars set in Vercel dashboard
Rollback	Vercel Instant Rollback to any previous deployment via dashboard; < 60 seconds to revert
Cron Jobs	Defined in vercel.json: generate-alerts runs daily at 01:00 UTC; update-aging called manually or via POST
Feature Flags	Not implemented in v1.0; controlled by code deployment (branch-based)

CI/CD Pipeline

Step	Action
1. Push	Developer pushes to feature branch on GitHub
2. PR Created	Vercel creates preview deployment automatically; URL posted to PR
3. Lint	GitHub Action runs ESLint (eslint-config-next); fails PR if errors found
4. Type Check	tsc --noEmit run in CI; TypeScript errors block merge
5. E2E Tests	Playwright tests run against Vercel preview deployment URL
6. Code Review	Minimum 1 approval required before merge to main
7. Merge to main	Vercel triggers production deployment automatically
8. Health Check	Post-deploy ping to /api/health; alert if unhealthy response
9. Monitor	Vercel Analytics dashboard checked for error spikes post-deploy

Risks, Timeline & Future Roadmap

Risk Register

ID	Category	Impact	Likelihood	Description	Mitigation
R-01	Technical	High	Medium	MongoDB Atlas connection drops under Vercel cold starts	Connection pooling via db.ts caching; retry logic on first connect; health check monitoring
R-02	Technical	Medium	Medium	Cron job exceeds 60s Vercel function timeout as company count grows	Optimize per-company processing with early exit on errors; consider batching companies if > 20
R-03	Security	High	Low	JWT_SECRET exposure in compromised environment	Rotate secret immediately via Vercel env var update; all existing tokens invalidated automatically
R-04	Operational	Medium	Medium	Gmail SMTP rate limits hit during bulk alert emails	Move to dedicated email service (Resend/SendGrid) in v2; current v1.0 acceptable for SMB scale
R-05	Business	High	Medium	Low adoption due to onboarding friction	Demo account available; onboarding wizard designed for < 10 min completion; no credit card required
R-06	Technical	Low	Low	Supabase permission table out of sync with MongoDB assignedWarehouses	seedUserPermissions() called atomically on both onboarding and invitation accept; manual sync script available
R-07	Security	Medium	Low	Cloudinary upload URL forgeable to upload malicious files	MIME validation + size limit enforced server-side before upload; no executable file types accepted
R-08	Business	Low	High	Dead stock thresholds (120/180 days) don't fit all industries	SystemConfig.agingThresholds exists but unused by cron; v2 will wire these to the aging engine

Milestone Timeline (Estimated)

Milestone	Timeline	Deliverables
M-1	Week 1-2	Core auth, onboarding wizard, user invitation flow
M-2	Week 3-4	Product and warehouse CRUD; stock entry and exit
M-3	Week 5-6	Stock transfer; aging engine; daily cron jobs
M-4	Week 7-8	Alert system (all 5 types); alert lifecycle management
M-5	Week 9-10	Notifications; email alerts; aging report; dashboard aggregation
M-6	Week 11-12	UI polish: dark mode, responsive, empty/loading/error states
M-7	Week 13-14	Security hardening; E2E testing; staging deployment
M-8	Week 15	Production deployment; monitoring setup; documentation

v2 Future Enhancements Roadmap

Feature	Description
Custom aging thresholds per company	Wire SystemConfig.agingThresholds to the aging engine instead of hardcoded 120/180
XYZ Analysis Report	Classify products as X (fast), Y (medium), Z (slow) based on movement frequency
Valuation & Damage Reports	Full inventory value over time; damage tracking with write-off records
Custom Report Builder	User-defined filters, columns, date ranges; export to PDF and Excel
Barcode/QR Scanning	Mobile scan for product lookup and stock entry/exit on the warehouse floor
SMS Notifications	WhatsApp/SMS alerts for critical events via Twilio or similar
ERP Integration	Bidirectional sync with QuickBooks, Tally, or Zoho Books for purchase orders
OAuth / SSO	Google and Microsoft sign-in; SAML for enterprise customers
MFA	TOTP or SMS-based multi-factor authentication for super_admin accounts
Rate Limiting	Per-IP and per-user rate limiting on all API endpoints
Subscription Billing	Stripe-based premium tiers for advanced features (custom alerts, API access, more users)
Real-time Updates	WebSocket or Server-Sent Events for live dashboard updates without page refresh
AI Demand Forecasting	Predict reorder points and seasonal demand using movement history

Feature	Description
Native Mobile App	React Native app for iOS and Android (primarily for warehouse staff on floor)

Glossary

Term	Definition
Batch	A single stock entry event. Each batch has a unique batchId, a quantity, an entry date, and a status. Transfers create new batches with a parentBatchId linking to the source.
parentBatchId	The batchId of the original stock batch from which a transfer was made. Used to trace the full lineage of stock from first entry through all transfers.
companyId	The ObjectId of the SystemConfig document representing a company. Present on every core entity as the multi-tenancy isolation key.
Dead Stock	Stock with ageInDays >= 180. Generates a Critical alert and deducts 15 points from the health score.
At-Risk Stock	Stock with ageInDays 120-179. Generates a Warning alert and deducts 5 points from the health score.
Health Score	Dashboard metric: $100 - (\text{deadCount} \times 15 + \text{atRiskCount} \times 5)$, minimum 10. Represents overall inventory quality.
Reorder Level	Product-level threshold for low-stock alerts. When quantityAvailable across a warehouse drops below this number, a low_stock alert is generated.
Supabase Permissions	The user_warehouse_permissions PostgreSQL table in Supabase that is the enforcement source of truth for which users can access which warehouses.
CRON_SECRET	The bearer token that secures all /api/cron/* endpoints. Must be set in production via environment variables. Requests without it return 401.
Aging Concern	A ProductCategory attribute: slow moderate fast expiry. Indicates how quickly products in that category are expected to move and what kind of aging alert applies.
StockMovement	An immutable audit record created for every stock transaction (entry, exit, transfer, damage, adjustment). Never updated after creation.
SRS	Software Requirements Specification — this document.
RBAC	Role-Based Access Control. The system that restricts what actions each role (super_admin, warehouse_manager) can perform.
SMB	Small-to-Medium Business. The primary target customer segment for Inventory Tracker.
TTL	Time-To-Live. The duration for which cached data (Next.js unstable_cache) remains valid before a fresh database query is made.
JWT	JSON Web Token. A signed, compact token carrying userId, email, role, and companyId. Stored as an HttpOnly cookie.

Key Business Rules Reference

Quick-reference summary of all 12 business rules from §3:

Rule	Description
BR-01	All company data isolated by companyId — no cross-company access at any layer
BR-02	Warehouse managers can only act on explicitly assigned warehouses (Supabase-enforced)
BR-03	Stock exit quantity cannot exceed quantityAvailable in the selected batch
BR-04	Warehouse with active stock cannot be deleted
BR-05	Product SKUs must be unique within a company; reusable across companies
BR-06	Stock transfer cannot exceed destination warehouse remaining capacity
BR-07	Invitation tokens expire after 3 days; password reset tokens after 1 hour
BR-08	Duplicate pending invitations to the same email are blocked
BR-09	Alert and notification creation is always non-blocking — stock operations succeed even if alerts fail
BR-10	Archived products (isActive=false) cannot receive new stock entries
BR-11	Signup endpoint permanently disabled (410); new users via onboarding or invitation only
BR-12	Cron endpoints secured by CRON_SECRET bearer token; not browser-accessible without it

Success Metrics

Metric	Target	Measurement
Onboarding Completion Rate	> 80%	% of users who start wizard and complete all 5 steps
Alert Resolution Rate	> 70%	% of critical alerts acknowledged within 24 hours
Dead Stock Reduction	>= 30%	After 6 months of use vs. pre-adoption baseline
API Error Rate	< 0.5%	% of requests returning 5xx status codes
Dashboard Load Time (p95)	< 2 seconds	Time to fully interactive dashboard on cold load
Time to First Stock Entry	< 10 minutes	From onboarding complete to first batch recorded
Daily Active Companies	> 60%	Companies with at least one stock operation per week

Disaster Recovery

Component	Backup Strategy	RPO (Max Data Loss)	RTO (Recovery Time)
MongoDB Atlas	Automated daily backups with 7-day retention; point-in-time recovery on M10+	< 24 hours	< 4 hours

Component	Backup Strategy	RPO (Max Data Loss)	RTO (Recovery Time)
Supabase PostgreSQL	Managed automated backups; restore via Supabase dashboard	< 24 hours	< 4 hours
Cloudinary Images	Managed by Cloudinary SLA; no additional backup needed	< 1 hour	< 30 min
Application Code	Git repository; any commit can be redeployed to Vercel in < 5 minutes	0 (code in Git)	< 5 minutes
Environment Variables	Stored in Vercel dashboard; must be manually documented in secure vault	N/A	< 30 min to reconfigure